Uploading from Devices

How files get into instmon's library from outside the node: a file uploaded from a phone or laptop, a file copied from a USB stick, a manual, and a configuration snapshot. It covers the checks on each route, the limits, and what is refused.

BROWSER UPLOAD · USB IMPORT · MANUALS · CONFIG SNAPSHOT

SUMMARY · READ-ONLY REVIEW

How a file gets in

A file from a device goes into one of four places. Each route has its own checks, and all of them stage the file into the library, so an install is a separate later step.

ROUTE	WHERE IT GOES	WHO USES IT
Browser upload (`/api/upload`)	The library folder for a chosen category, or Manuals for a PDF	A phone, tablet or laptop, through the page
USB import (`/api/usbimport/import`)	The same library folders, copied from a stick	A USB drive plugged into the node
Config snapshot (`/api/confputter/upload`)	A holding folder for Confputter, not the library	A Config getter snapshot, edited on a device
Editor save (`/api/file`)	The file already in the library, edited on the node	The page's editor (not an upload)

Staging a file does not install it. The install is started separately from the page, through the component's own `--install` (Part 3).

Uploading through the page

2.1 The request

The page sends a multipart form with three fields: `category` (one of the library categories), `file` (the file), and an optional `force` flag set to "1". The request must be multipart form data and must include a Content-Length. A request with no length, or a length over the limit, is refused with 400.

LIMIT	DEFAULT	SET BY
Upload size	50 MB	INSTMON_MAX_UPLOAD_MB
Manual (PDF) size	64 MB	Fixed in the code. Reached only by the USB import. A browser upload is checked against the 50 MB request limit first, so a PDF over 50 MB cannot be uploaded through the page

The whole request body is read into memory before it is parsed, so the limit also bounds the memory used by one upload.

2.2 The checks, in order

1. **Category.** The category must be a known library category, or the upload is refused.
2. **File name.** The name is reduced to its base name, so no folder can be given.
3. **Extension.** The extension must be allowed for the category. The table in Part 3 lists them.
4. **Name pattern.** The name must match a pattern for the category (for example `wifimon*.py` for wifimon). A name that does not match is refused with "doesn't look like a … file", unless `force` is set. The refusal is marked as a mismatch, so the page can offer to force it.
5. **Lock.** If the library file is locked (a locked file is one an operator has put aside) and the new content differs from it, the upload is refused with 409. An identical file is not an overwrite and goes through to the next step.
6. **Write.** The file is written only if its content differs from the copy in the library. An identical file is reported as already staged.

A forced mismatch is logged with the word "forced", so the book can point to it when a file is in the wrong place.

2.3 PDFs

A PDF is always a manual, whatever category was chosen: a file whose name ends in .pdf is routed to Manuals before the category is looked at. The manual's name is cleaned (anything other than letters, digits, `.`, `_` and `-` becomes `_`, at most 120 characters), and the data must start with `%PDF-`, or it is refused as not a PDF. A manual identical to the one already there is reported as nothing to do. The file is written to a temporary name and renamed, with mode 644. In the other direction, an upload under the manuals category that is not a PDF is refused: "Only PDF files go in Manuals."

What each category accepts

CATEGORY	EXTENSIONS	NAME PATTERNS
Dashboard	.py	nodecontrol*.py, asl_dvs_dashboard*.py
SysMon	.py	sysmon*.py, asl_dvs_sysmon*.py, asl_dvs_m17_sysmon*.py, sysmon_svx*.py
wifimon	.py	wifimon*.py
44helper	.py	44helper*.py, asl_dvs_m17_44helper*.py
Node Talker	.py	nodetalker*.py, nodelisten*.py
Node Recorder	.py	noderecorder*.py
Pibrary	.py	pibrary*.py
instmon	.py	instmon*.py
Scripts	.sh	install_all*.sh, uninstall_all*.sh, install_asl_dvs*.sh, uninstall_asl_dvs*.sh, wifi_menu*.sh, wifi-menu*.sh, asl_dvs*.sh
Config files	.conf	asl_dvs*.conf, sysmon*.conf, wifimon*.conf, nodetalker*.conf, nodelisten*.conf
Confgetter	.sh	confgetter*.sh
Confputter	.py	confputter*.py

The name pattern is a shell-style match, ignoring case. A file named for a different tool, or with no matching name, is refused unless forced. In the code, the Node Talker category is named `nodelisten` (its library folder too), and it accepts both `nodetalker*.py` and `nodelisten*.py`.

Copying from a stick

4.1 Choosing the drive

The page lists the drives that `lsblk` reports as whole disks with a positive size, and that are on a USB connection or flagged removable. The boot disk is marked and refused for import. The drive name is checked against a safe pattern (letters, digits, `_`, `.` and `-`) before anything else.

4.2 Mounting, read-only

1. The import waits its turn: it is refused while a disk-image job is running (answer 409), and only one import can run at a time.
2. The partition is found. If it is already mounted somewhere, that mount point is used and is not unmounted afterwards.
3. Otherwise it is mounted under `/mnt/instmon-usbimport/<drive>` with the options `ro,nosuid,nodev,noexec`, and `noload` for ext3 and ext4. A read-only mount means the stick cannot be changed by the import.
4. File systems: vfat, exfat, ext2, ext3 and ext4 are mounted as they are. NTFS needs the ntfs3 driver in the kernel or the ntfs-3g helper; if neither is there, the mount is refused with a message to install ntfs-3g.
5. When the import ends, a mount made by instmon is unmounted, whatever the outcome.

4.3 Browsing and choosing a file

The browse call lists the folder on the stick, up to 500 entries, showing only files of the allowed types and the manual type. Skipped are names that start with a dot or a dollar sign, "system volume information", "lost+found", "$recycle.bin", and every symbolic link. Every path is resolved inside the mount point, so a path that climbs out of it is refused.

4.4 Importing one file

- The file must be a regular file. It is opened without following links and without blocking, so a device file or a symbolic link is refused.
- The size is checked twice: once from the file's metadata, and again as the data is read. A file over the limit (50 MB, or 64 MB for a manual) is refused.
- A PDF goes to Manuals. Any other file goes through the same checks as a browser upload (Part 2.2), with the source marked as USB. The log then says "Imported from USB".

Uploading a configuration for Confputter

Confputter is a separate tool that applies a configuration. Its input is a snapshot written by Confgetter, a Markdown file. A snapshot edited on a laptop can be uploaded back to the node through this route.

- **Name.** The name must start with a letter or digit, use only letters, digits, `.`, `_` and `-`, be at most 101 characters before the ending, and end in `.md`.
- **Size.** No more than 3,000,000 bytes, and not empty. The request itself may be 64 KB larger, for the form wrapper.
- **Text.** The file must be UTF-8 text.
- **Heading.** The first 40 lines must include a line that begins `# Node config snapshot:`. A file without it is refused as not a Config getter snapshot.
- **Where it goes.** The edited-snapshots folder, which is kept with mode 700. The file is written to a temporary name and renamed into place, with mode 600.

The upload does not apply anything. Applying is a separate Confputter step.

Saving an edit on the node

The page's editor saves a change to a file that is already in the library. It is not an upload, but the checks are the same kind:

- The file must already exist in the library (a file that is not there is refused with 404).
- A locked file is refused with 409 whatever the new content is, and the log records the refusal. This is stricter than the upload route, which refuses a locked file only when the content differs.
- A Python (.py) or shell (.sh) file is checked for syntax before it is saved. A file with a syntax error is not saved.

What the upload routes show

7.1 Staging is not installing

Every route ends in the library, and nothing runs. The book can describe the library as a waiting room between a file's arrival and its install.

7.2 The name rules do the most work

A browser upload is stopped mostly by its name. A file with the wrong name goes nowhere unless forced, and the force is logged. A forced file of the wrong kind can still be installed later, so operators should check the name before they force anything.

7.3 The USB stick is read-only

The stick is mounted read-only and without execution, and its files are copied, not run. The book can say that plugging in a stick cannot change it, or run anything from it.

7.4 Not checked

- The browser's side of the upload (the form and the force prompt), which was read for the request shape only.
- What Confputter does with an uploaded snapshot once it is applied.
- The kernel's NTFS driver behaviour on a real stick.